An Outlier Mining Algorithm Based on Dissimilarity

Matheus Peixoto Ribeiro Vieira - 22.1.4104
Pedro Henrique R. L. de Oliveira - 22.1.4022

Outliers

“É uma observação que desvia tanto de outras observações a ponto de sugerir que os dados foram medidos por outro mecanismo”

São gerados por:

- Natureza própria dos dados.
- Erros de execução manual ou falhas de equipamentos.
- Dados colocados em classes erradas .

Outliers

A mineração de outliers busca encontrar dados “excepcionais, interessantes, esparsos e padrões isolados” no conjunto de dados

Aplicações:

- Transações monetárias suspeitas.
- Detecção de invasão em redes.
- Verificação de empréstimos.

Diferentes métodos

Baseados em estatística: Usam testes inconsistentes para determinar os outliers com uma distribuição de probabilidade conhecida do conjunto de dados

Necessitam de conhecimento prévio da distribuição dos dados.

Baseados em distância: Calculam a distância entre todos os pontos. Os outliers são aqueles que não possuem vizinhos o suficiente.

A utilização destes métodos requer a identificação de parâmetros adequados.

Diferentes métodos

Baseados em densidade: Compara a densidade da área ao redor de um objeto com a dos seus vizinhos. Se a densidade for menor que um limiar, então o dado é considerado um outlier.

Baseado em desvio: Verifica se os dados se enquadram em uma lista de características criadas. Se não se encaixa, então é um outlier.

Definições formais (Dataset)

D é um dataset formado por objetos e atributos.

$$D = (U, A)$$

U são os **m** objetos.

A são os **n** atributos.

Definições formais (Dissimilaridade de atributos)

$$ad_{ij}^{f} = \frac{\left\| x_{if} - x_{f} \right| - \left| (x_{jf} - x_{f}) \right\| * \left\| x_{if} - x_{f} \right| - \left| x_{jf} - x_{f} \right\|}{x_{\max f} - x_{\min f}}$$

Seja f ∈ A, u_i e u_j ∈ U:

$X_{\alpha f}$ é o valor do atributo f do objeto u_{α}.

X_f é o valor médio do atributo f.

$X_{max\ f}$ é o maior valor do atributo f.

$X_{min\ f}$ é o menor valor do atributo f.

Definições formais (Dissimilaridade de objetos)

$$od(i, j) = \frac{\sum_{k=1}^{n} ad_{ij}^{a_k}}{n}$$

n é o número de atributos.

Definições formais (Matriz de dissimilaridade)

$$dm = \begin{vmatrix} od(1,1) & od(1,2) & \cdots & od(1,m-1) & od(1,m) \\ od(2,1) & od(2,2) & \cdots & od(2,m-1) & od(2,m) \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ od(m-1,1) & od(m-1,2) & \cdots & od(m-1.m-1) & od(m-1,m) \\ od(m,1) & od(m,2) & \cdots & od(m,m-1) & od(m,m) \end{vmatrix}$$

Definições formais (Dissimilaridade de sinergia)

$$sd_i = \sum_{j=1}^{m} od(i, j)$$

Definições formais (Dissimilaridade máxima)

$$d_{\max} = \max_{i=1}^{m} sd_i$$

Definições formais (Grau de dissimilaridade)

$$dd_i = \frac{d_{\max} - sd_i}{d_{\max}}$$

Quanto menor o dd_i, maior a chance de u_i ser um outlier.

Algoritmo

```
1  def omabd(x: list, tdd:float) -> list:
2      """
3      Input:
4          1) x: Dataset D representado por x[0..m][0..n],
5              onde x[i][j] é o j-ésimo atributo do i-ésimo objeto
6          2) tdd: Limiar de dissimilaridade
7
8      Retorno:
9          Índices dos outliers
10     """
```

Algoritmo

```python
# Passo 1: Obter Xmaxf, Xminf e Xf de cada atributo
m = len(x)
n = len(x[0])

xmaxf = [0 for _ in range(n)]
xminf = [0 for _ in range(n)]
xf    = [0 for _ in range(n)]

for i in range(n):
    xmaxf[i] = x[0][i]
    xminf[i] = x[0][i]
    xf[i] = x[0][i]

    for j in range(1, m):
        if x[j][i] > xmaxf[i]: xmaxf[i] = x[j][i]
        if x[j][i] < xminf[i]: xminf[i] = x[j][i]
        xf[i] += x[j][i]

    xf[i] /= m
```

Algoritmo

```
# Passo 2: Computar a matriz dm e dmax
dmax = 0.0
sd = [0 for _ in range(m)]
od = [[0 for _ in range(m)] for _ in range(m)]

for i in range(m):
    for j in range(m):
        for k in range(n):
            temp = abs(abs(x[i][k]-xf[k]) - abs(x[j][k]-xf[k]))
            od[i][j] += (temp*temp) / (xmaxf[k]-xminf[k])

        od[i][j] = od[i][j] / n
        sd[i] += od[i][j]

        if dmax < sd[i]: dmax=sd[i]
```

Algoritmo

```
# Passo 3: Detectando outliers usando o tdd
outliers = []
for i in range(m):
    dd = (dmax - sd[i]) / dmax
    if dd < tdd: outliers.append(i)

return outliers
```

Algoritmo

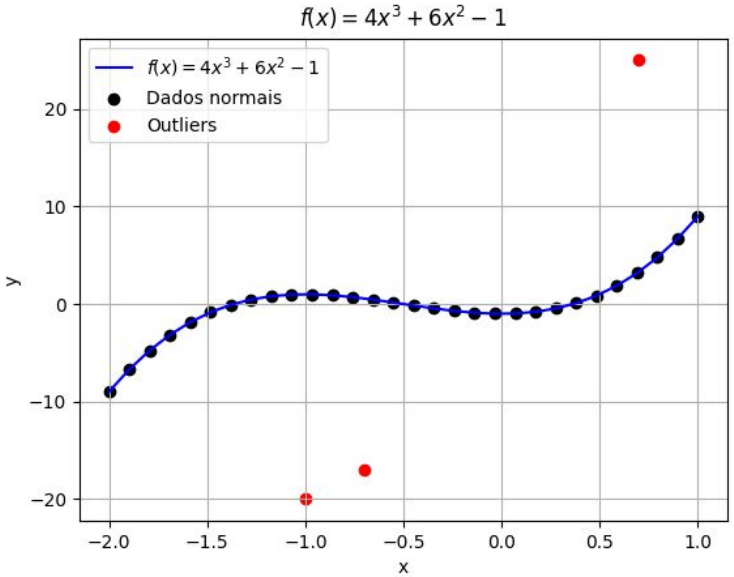
```python
# Definindo a função polinomial
coefficients = [4, 6, 0, -1]
polynomial = np.poly1d(coefficients)

# Gerando os valores de X e Y
x = np.linspace(-2, 1, 30)
y = polynomial(x)

x = x.tolist()
y = y.tolist()

# Gerando outliers arbitrários
outliers = [
    (-1.0, -20.0),
    (-0.7, -17.0),
    (0.7, 25.0)
]

D = list(zip(x, y)) + outliers
D = sorted(D, key=lambda x: x[0])
```




```python
indexes = omabd(D, tdd=0.6)
print(f"outliers: {indexes}")
for i in indexes:
    print(D[i])
```

✓ 0.0s

```
outliers: [10, 14, 29]
(-1.0, -20.0)
(-0.7, -17.0)
(0.7, 25.0)
```

Complexidade do algoritmo

É afetado pela quantidade de objetos (**m**) e de atributos (**n**)

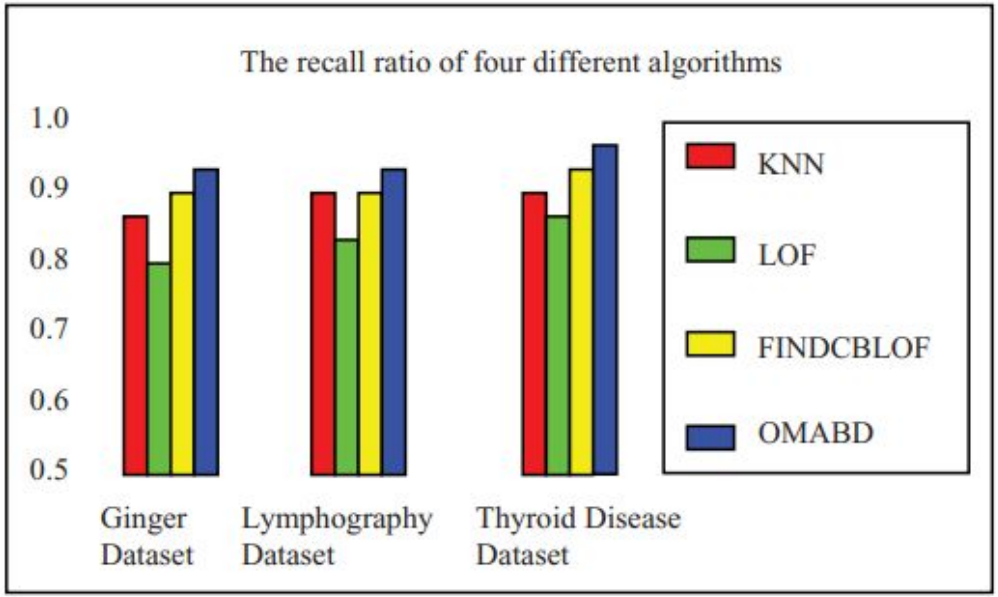
A complexidade de cada passo é:

- Obter X_{maxf}, X_{minf} e X_f: $O(m*n)$
- Computar a matriz de dissimilaridade: $O(m^{2}*n)$
- Detectando outliers: $O(m)$

Complexidade final:

$$O(m*n + m^{2}*n + m) \text{ --> } O(m^{2}*n)$$

Experimentos



Experimentos

TABLE I THE precision ratio WITH DIFFERENT DATASET

	KNN	LOF	FINDCBLOF	OMABD
ginger dataset	0.88	0.81	0.90	0.93
Lymphography dataset	0.89	0.84	0.90	0.94
Thyroid Disease dataset	0.91	0.89	0.92	0.96

Referências

Zhou, M.-J. & Chen, X.-J. 2012. An outlier mining algorithm based on dissimilarity. Procedia Environmental Sciences 12, 810–814.